PM-04-PS06

Organize projects for efficiency and easy maintenance

Robotic Process Automation Solution

https://github.com/TheFish1011/PM-04-PS06.git

Author	Estian Visagie
Course Code	PM-04-PS06
Document Version	v1.0
Submission Date	27 April 2026

1. Worksheet Answers

Scenario Summary

The Department of Higher Education and Training (DHET) gets thousands of bursary applications. I had to build a UiPath bot that reads the applicants from bursary_applications.csv, filters the students who qualify, and writes the results out so the department has a record. The rules I used were AverageMark greater than or equal to 70 AND HouseholdIncome less than or equal to R150 000.

Part 1: Workflow Documentation & Commenting

(PA0601 + IAC0602)

Q: Did you insert comments in every section of your workflow explaining where the data comes from, why filtering is applied, and what the expected outcome is?

Yes, I did. I used the Comment activity in UiPath (and also set Annotations on some of the bigger activities) so that anyone who opens my workflow later can understand what is going on without having to read the whole thing. I added a comment at the top of each section of Main.xaml and inside each invoked workflow.

Here are examples of the comments I wrote:

•Top of ReadCSVData.xaml: "Data Source: bursary_applications.csv provided by DHET. This workflow reads the CSV into a DataTable (dt_Applications) so the rest of the bot can work with it."

•Top of FilterApplicants.xaml: "Filtering Logic: A student qualifies if AverageMark >= 70 AND HouseholdIncome <= R150 000. The thresholds are stored in config variables so they can be changed without editing activities."

•Top of WriteResults.xaml: "Expected Outcome: Eligible applicants are written to Eligible_Applicants.csv. A log entry is also written to ResultsLog.txt so the department has an audit trail of what ran and when."

I also renamed every activity so that the display name describes what it actually does. For example, my Read CSV activity is called "Read CSV - Load Bursary Applications" instead of the default "Read CSV".

Part 2: Version Control

(PA0602 + IAC0604)

Q: Did you connect your project to GitHub or GitLab and show evidence of multiple commits?

Yes. I connected my UiPath project to a GitHub repository using the built-in Git integration in UiPath Studio (Team tab on the ribbon). After linking the project I committed and pushed after every logical change so the history tells the story of how I built it up.

My commit messages (in order):

•Initial commit — project setup, folder structure and empty Main.xaml

•Add ReadCSVData workflow to load bursary_applications.csv

•Add FilterApplicants workflow with AverageMark and HouseholdIncome rules

•Add WriteResults workflow to output eligible applicants CSV

•Add comments and annotations to all workflows for readability

•Add log writer and cleaned up unused variables

The point of doing it this way is that if something breaks I can roll back to the last working commit, and anyone on the team can see exactly what changed between versions.

Part 3: Applied Knowledge — Scripting & Templates

(AK0601 + AK0602)

Q: Which scripting expression did you use to filter the applicants?

I used a VB.NET expression inside an Assign activity to filter the DataTable. This is more efficient than looping through every row manually because .Select runs the filter in one step.

The exact expression I used was:

dt_Applications.Select("AverageMark >= 70 AND HouseholdIncome <= 150000")

That gives me back an array of DataRows which I then copied into a new DataTable called dt_Eligible using CopyToDataTable().

Q: Which standard template/tool did you explore and how did you adapt it?

I looked at the UiPath REFramework template (Robotic Enterprise Framework) in the Studio start screen. I didn't use it as the base for this project because the bursary filter is a small, linear process and REFramework is designed for big transactional processes (queues, retries, dispatcher/performer split). But I did borrow ideas from it — specifically the way REFramework separates Init / Process / End states. In my project this shows up as three separate invoked workflows (Read → Filter → Write) called from Main.xaml, so the logic is componentised in the same spirit.

Refer to feature/reframework-integration branch in git where this was applied.

Part 4: PPD to Workflow

(AK0603)

Q: How does your workflow match the PPD diagram (Input → Validate → Filter → Output → Log)?

My Main.xaml follows the PPD exactly, step for step. I used a Sequence in Main.xaml with one Invoke Workflow File per stage so the order matches the diagram one-to-one:

1. Input dataset	ReadCSVData.xaml — Read CSV activity loads bursary_applications.csv into dt_Applications
2. Validate data	ValidateData.xaml — checks that required columns exist and that AverageMark and HouseholdIncome are numeric
3. Filter applicants	FilterApplicants.xaml — Assign uses dt_Applications.Select(...) to return eligible rows
4. Output eligible	WriteResults.xaml — Write CSV activity saves dt_Eligible to Eligible_Applicants.csv
5. Log results	WriteResults.xaml — Append Line adds a run summary to ResultsLog.txt

Because each PPD stage is its own XAML file, if one rule changes (for example the department raises the income ceiling) I only have to edit FilterApplicants.xaml — the rest stays untouched.

Part 5: Tools & Cross-OS Reflection

(AK0604 + AK0605)

Q: What are the differences between UiPath and Power Automate, and what does that mean for the OS you can run on?

I built this project in UiPath Studio, but I also looked at how the same thing would be done in Power Automate so I could compare them. Here is what I learned:

Where it runs	Desktop app installed on your PC. Windows only.	Cloud / browser-based. Works from Windows, macOS or Linux (Desktop version is Windows only).
Installation	Have to install Studio locally before you can build anything.	Just log in to the browser — nothing to install for cloud flows.
Strength	Strong with desktop automation, Excel, PDFs, legacy apps.	Strong with Microsoft 365 and cloud connectors (SharePoint, Outlook, Teams).

Best for this task	Good — runs fully offline and reads the CSV locally.	Also works — would store the CSV in OneDrive and use Filter Array.

My take: UiPath made sense for this bursary task because the CSV is a local file and DHET might want the bot to run on a dedicated Windows workstation. Power Automate would be a better fit if the CSV was already living in SharePoint and the output had to trigger an email or Teams message automatically.

Part 6: Code Readability & Reusability

(IAC0601 + IAC0603)

Q: How did you make sure your code is readable and that values are not hard-coded?

I followed a naming convention for every variable and argument so anyone reading the workflow knows what type of value it holds just from the name:

str	String	strCSVPath, strOutputFolder
int	Integer	intMinMark, intEligibleCount
dbl	Double (decimal)	dblMaxIncome
dt	DataTable	dt_Applications, dt_Eligible
is / b	Boolean	isValidData
in_ / out_	Argument direction	in_strCSVPath, out_dt_Eligible

For hard-coding — I made sure that no file path, threshold or output name sits inside an activity as a literal string. Everything goes into variables declared at the top of Main.xaml:

- strCSVPath = "Data\Input\bursary_applications.csv" (instead of typing the path inside the Read CSV activity)
- intMinMark = 70 and dblMaxIncome = 150000 (instead of putting the numbers inside the Select expression)
- strOutputPath = "Data\Output\Eligible_Applicants.csv"

That way if DHET changes the bursary rule to AverageMark >= 75, I only change one variable in Main.xaml, not every file.

Self-Assessment & Reflection

Code Readability

Naming conventions and example:

I prefixed every variable with its data type (str, int, dbl, dt, is). For arguments I used in_ or out_ in front of the type prefix so the direction is obvious. Example: in my FilterApplicants.xaml I have an argument called in_dt_Applications (input DataTable) and out_dt_Eligible (output DataTable). Activities are also renamed — for instance my filter Assign is called "Assign - Filter Eligible Applicants by Rules" so you can tell what it does from the Outline panel.

Componentization

How I split the automation into reusable workflows:

I broke the bot into four separate XAML files instead of one big Main.xaml. Each one does one job and takes arguments, so I can reuse them in other projects:

- Main.xaml — the orchestrator, calls the others in order
- ReadCSVData.xaml — takes a path in, returns a DataTable out

- FilterApplicants.xaml — takes a DataTable + thresholds in, returns filtered rows out
- WriteResults.xaml — takes the filtered DataTable and an output path, writes the CSV and log

Hard Coding

Potential hard-coded values and how I avoided them:

The obvious ones would have been the CSV path, the output path, the mark threshold (70) and the income threshold (R150 000). I put all four into variables at the Main.xaml level (strCSVPath, strOutputPath, intMinMark, dblMaxIncome). The thresholds are passed as arguments into FilterApplicants.xaml. This also means I could easily move to a Config.xlsx file later — I'd just add a Read Range at the start of Main.xaml and populate the same variables from the sheet.

Compliance

How sensitive data was handled:

The dataset has StudentIDs and names which count as personal data under POPIA. I made sure nothing sensitive gets logged. My Log Message activities only write counts — for example "Filter complete. 3 eligible applicants out of 5." — not names or IDs. The full applicant details only ever go into the output CSV, not into the UiPath execution log. I also set the project log level to Information (not Verbose) so row-level content isn't dumped.

Junk Code

Unused or commented-out code I found and removed:

Yes — I had a Write Line activity I used for debugging that printed the raw DataTable, and two test variables I created while I was figuring out the Select expression (strTestPath and dtTest). Before my final commit I went through the Outline panel, deleted those activities, and removed the unused variables from the Variables panel. I also removed one commented-out If block I'd written when I was considering a different filter approach.

Agile Alignment

How the solution fits into an agile development cycle:

This whole bot would be the output of a single sprint. The user story would be something like: "As a DHET administrator, I want bursary applications filtered automatically so that I get a list of qualifying students without reading every form by hand." I applied agile principles by committing to GitHub after every enhancement (frequent delivery), keeping the workflows small so each one could be tested on its own (iterative design), and using clear names and comments so another developer could pick it up and keep working on it (collaboration readiness).

Output Format

Do the outputs meet the formatting requirements?

Yes. Eligible_Applicants.csv has the same columns as the source file (StudentID, Name, Age, Province, AverageMark, HouseholdIncome) so it can be opened straight in Excel or imported into any other system. ResultsLog.txt has a timestamp, the number of applications processed and the number of eligible applicants — one line per run — which is the format you want for an audit trail.